

飞腾向量基础数学库用户指南

V1.3 (2025-03-27)

版权所有 © 飞腾信息技术有限公司 2022。保留所有权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium和其他飞腾商标均为飞腾信息技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

本文档的内容视为飞腾的保密信息，您应当严格遵守保密义务；未经飞腾事先书面同意，您不得向任何第三方披露本文档内容或提供给任何第三方使用。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，飞腾在现有技术的基础上尽最大努力提供相应的介绍及操作指引，但飞腾在此明确声明对本文档内容的准确性、完整性、适用性、可靠性等不作任何明示或暗示的保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

本文档中所有内容，包括但不限于图片、架构设计、页面布局、文字描述，均由飞腾和/或其关联公司依法拥有其知识产权，包括但不限于商标权、专利权、著作权等。非经飞腾和/或其关联公司书面同意，任何人不得擅自使用、修改、复制上述内容。
如有技术问题，可联系 support@phytium.com.cn 获取支持。

飞腾信息技术有限公司

地址：天津市滨海新区海缘路1号信安创业广场5号楼

网址：www.phytium.com.cn

版本历史

文档版本	发布日期	更新说明
V1.0	2024-03-22	首次发布
V1.1	2024-06-26	版本更新
V1.2	2024-09-30	版本更新
V1.3	2025-03-27	增加测试程序

目 录

1 引言	1
2 安装指南	1
2.1 环境要求	1
2.1.1 硬件要求	1
2.1.2 软件要求	1
2.2 VML-FT 的安装和使用	2
2.2.1 部署 VML-FT	2
2.2.2 使用 VML-FT	2
2.2.3 测试	3
3 VML-FT 函数说明	1
3.1 接口组成	1
3.2 函数命名	2
3.3 函数定义	3
3.3.1 三角函数	3
3.3.2 双曲函数	7
3.3.3 指数对数函数	10
3.3.4 特殊函数和随机函数	16
3.3.5 比较函数和最值函数	18
3.3.6 基本运算函数	23
3.3.7 其它函数	37

表 目 录

表 2-1 软件环境 1

表 3-1 VML-FT 接口组成表 1

表 3-2 数据类型表 2

表 3-3 函数名表 2

表 3-4 数据精度表 2

表 3-5 VML-FT 函数命名示例表 2

1 引言

飞腾向量基础数学库 VML-FT (Vector Math Library for Phytium) 库包括三角函数、幂函数、双曲函数、指数函数、对数函数等，基于飞腾 CPU 体系结构特点和指令集特点，通过向量化、算法改进等优化方法，实现了在飞腾平台上的性能提升。

2

安装指南

本章介绍了如何在飞腾平台上安装使用 VML-FT 库。

2.1 环境要求

2.1.1 硬件要求

飞腾向量基础数学库紧密支持飞腾处理器，已在 FT-2000/4、飞腾腾锐 D2000、FT-2000+/64、飞腾腾云 S2500、飞腾腾云 S5000C 处理器上成熟运用。

2.1.2 软件要求

软件环境如表 2-1 所示。

表 2-1 软件环境

项目	版本
OS	CentOS8.2、Kylin V10、Ubuntu19.10、 OpenEuler22.03
GCC	7.3.0、8.3.1、9.3.0、10.3.1、11.3.0
GNU Binutils	2.32、2.33、2.34、2.35、2.37
Glibc	2.28、2.23、2.31、2.30、2.34

2.2 VML-FT 的安装和使用

2.2.1 部署 VML-FT

步骤一：获取并解压缩 VML-FT.zip，解压缩目录下包含 VML-FT 的库文件和头文件；

步骤二：设置库路径

```
export LD_LIBRARY_PATH=$HOME/vml-ft/lib:$LD_LIBRARY_PATH
```

生成的库在解压缩文件的 lib 目录下。

2.2.2 使用 VML-FT

测试例子：以双精度向量加为例，测试代码 test.c 如下所示：

```
#include<stdlib.h>
#include<stdio.h>
#include<vml-ft.h>
void InitVal(int len, double *dst){
    double tmp;
    while(len--){
        *dst = ((double)(rand()%10000 - 5000)) / 10;
        dst++;
    }return;
}

int main(void)
{
    int N = 10;
    double *src1 = (double*)malloc(sizeof(double)*N),
           *src2 = (double*)malloc(sizeof(double)*N),
           *dst = (double*)malloc(sizeof(double)*N);
    InitVal(N, src1);
    InitVal(N, src2);

    vadd_d(N, src1, src2, dst);

    free(src1); free(src2); free(dst);
    return 0;
}
```

编译：

```
gcc test.c -I/$HOME/vml-ft/include -L/$HOME/vml-ft/lib -o test -lvml-ft -lm
```

运行：


```
./test
```

2.2.3 测试

步骤一：获取测试程序集 VML-FT_testing。

步骤二：编译测试程序：

```
make IDIR=PATH1 LDIR=PATH2
```

其中 PATH1 为 VML-FT 头文件所在目录，PATH2 为 VML-FT 库文件所在目录。

步骤三：设置库路径：

```
export LD_LIBRARY_PATH=PATH2:$LD_LIBRARY_PATH
```

其中 PATH2 为 VML-FT 库文件所在目录。

步骤四：运行：

```
./main
```

3 VML-FT 函数说明

3.1 接口组成

向量基础数学库 VML-FT 函数接口组成如表 3-1 所示：

表 3-1 VML-FT 接口组成表

接口类型	接口数量	说明
三角函数	22	求向量的正弦余弦值，正切余切值等
双曲函数	18	求向量的双曲正弦值，双曲余弦值等
指数对数函数	40	计算以自然常数 e，2,10 为底的指数(对数)等函数
特殊函数和随机函数	18	误差函数，随机向量生成等函数
比较，最值函数	38	比较向量的大小，求最大最小值等
基本运算函数	134	包括向量的加减乘除，求绝对值，求模运算等
其它函数	78	

3.2 函数命名

<数据类型>+<函数名> (+<数据类型>+<函数名>) + <数据精度>
<数据类型>说明，如表 3-2 所示：

表 3-2 数据类型表

<数据类型>	说明
v	实数向量
cv	复数向量
s	标量
rs	实数标量
vr	实数向量

<函数名>说明，如表 3-3 所示：

表 3-3 函数名表

<函数名>	说明
add	加法
sin	求正弦
exp	以自然常数 e 为底的 指数函数
...	...

<数据精度>说明，如表 3-4 所示

表 3-4 数据精度表

<数据精度>	说明
_d	双精度
_f	单精度

例子，以除法为例，如表 3-5 所示

表 3-5 VML-FT 函数命名示例表

函数名	说明
vdiv_d	双精度实数向量除以向量
vdiv_f	单精度实数向量除以向量
cvdiv_d	双精度复数向量除以向量
cvrdiv_d	双精度复数向量除以双精度实数标量

函数名	说明
rscvdiv_d	双精度实数标量除以双精度复数向量

3.3 函数定义

3.3.1 三角函数

3.3.1.1 Sin

函数定义

```
void vsin_f(const int len, const float *src, float *dst);
```

```
void vsin_d(const int len, const double *src, double *dst);
```

接口说明：vsin_f(vsin_d)计算输入单精度（双精度）向量的正弦值。

$$dst_i = \sin(src_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.2 Cos

函数定义

```
void vcos_f(const int len, const float *src, float *dst);
```

```
void vcos_d(const int len, const double *src, double *dst);
```

接口说明：vcos_f(vcos_d)计算输入单精度（双精度）向量的余弦值。

$$dst_i = \cos(src_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.3 Tan

函数定义

```
void vtan_f(const int len, const float *src, float *dst);  
void vtan_d(const int len, const double *src, double *dst);
```

接口说明：vtan_f(vtan_d)计算输入单精度（双精度）向量的正切值。

$$dst_i = \tan(src_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.4 Cot

函数定义

```
void vcot_f(const int len, const float *src, float *dst);  
void vcot_d(const int len, const double *src, double *dst);
```

接口说明：vcot_f(vcot_d)计算输入单精度（双精度）向量的余切。

$$dst_i = \cot(src_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.5 Csc

函数定义

```
void vcsc_f(const int len, const float *src, float *dst);  
void vcsc_d(const int len, const double *src, double *dst);
```

接口说明：vcsc_f(vcsc_d)计算输入单精度（双精度）向量的余割。

$$dst_i = \csc(src_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.6 Sec

函数定义

```
void vsec_f(const int len, const float *src, float *dst);
void vsec_d(const int len, const double *src, double *dst);
```

接口说明：vsec_f(vsec_d)计算输入单精度（双精度）向量的正割。

$$dst_i = sec(src_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.7 Asin

函数定义

```
void vasin_f(const int len, const float *src, float *dst);
void vasin_d(const int len, const double *src, double *dst);
```

接口说明：vasin_f(vasin_d)计算输入单精度（双精度）向量的反正弦值。

$$dst_i = asin(src_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.8 Acos

函数定义

```
void vacos_f(const int len, const float *src, float *dst);
```

```
void vacos_d(const int len, const double *src, double *dst);
```

接口说明：vacos_f(vacos_d)计算输入单精度（双精度）向量的反余弦值。

$$dst_i = acos(src_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.9 Atan

函数定义

```
void vatan_f(const int len, const float *src, float *dst);
```

```
void vatan_d(const int len, const double *src, double *dst);
```

接口说明：vatan_f(vatan_d)计算输入单精度（双精度）向量的反正切值。

$$dst_i = atan(src_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.10 Atan2

函数定义

```
void vatan2_f(const int len, const float *src1, const float *src2, float *dst);
```

```
void vatan2_d(const int len, const double *src1, const double *src2, double *dst);
```

接口说明：计算 src1/src2 的反正切值，src1 和 src2 和输出结果均为向量。

$$dst_i = atan2(src1_i, src2_i); \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.1.11 Sincos

函数定义

```
void vsincos_f(const int len, const float *src, float *sindst, float *cosdst);  
void vsincos_d(const int len, const double *src, double *sindst, double *cosdst);
```

接口说明：计算输入向量的正弦值和余弦值，输入输出均为向量。

$$\text{sindst}_i = \sin(\text{src}_i); \quad 0 \leq i < \text{len}$$

$$\text{cosdst}_i = \cos(\text{src}_i); \quad 0 \leq i < \text{len}$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
sindst	单(双)精度浮点类型	输出向量 sindst	输出
sosdst	单(双)精度浮点类型	输出向量 cosdst	输出

3.3.2 双曲函数

3.3.2.1 Sinh

函数定义

```
void vsinh_f(const int len, const float *src, float *dst);  
void vsinh_d(const int len, const double *src, double *dst);
```

接口说明：计算输入向量的双曲正弦值，输出为向量。

$$\text{dst}_i = \sinh(\text{src}_i); \quad 0 \leq i < \text{len}$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

函数定义

```
void cvsinh_f(const int len, const float *src, float *dst);  
void cvsinh_d(const int len, const double *src, double *dst);
```

接口说明：计算输入向量的双曲正弦值，输出为向量。

$$\text{dst}_i = \cosh(\text{src}_i); \quad 0 \leq i < \text{len}$$

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	复数单(双)精度浮点类型	输入向量 src	输入
dst	复数单(双)精度浮点类型	输出向量 dst	输出

3.3.2.2 Cosh

函数定义

```
void vcosh_f(const int len, const float *src, float *dst);
void vcosh_d(const int len, const double *src, double *dst);
```

接口说明: 计算输入向量的双曲余弦值, 输出为向量。

$$dst_i = \cosh(src_i); \quad 0 \leq i < len$$

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

函数定义

```
void cvcosh_f(const int len, const float *src, float *dst);
void cvcosh_d(const int len, const double *src, double *dst);
```

接口说明: 计算输入向量的双曲余弦值, 输出为向量。

$$dst_i = \cosh(src_i); \quad 0 \leq i < len$$

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	复数单(双)精度浮点类型	输入向量 src	输入
dst	复数单(双)精度浮点类型	输出向量 dst	输出

3.3.2.3 Tanh

函数定义

```
void vtanh_f(const int len, const float *src, float *dst);
void vtanh_d(const int len, const double *src, double *dst);
```

接口说明: 计算输入向量的双曲正切值, 输出为向量。

$$dst_i = \tanh(src_i); \quad 0 \leq i < len$$

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

函数定义

```
void cvtanh_f(const int len, const float *src, float *dst);
```

```
void cvtanh_d(const int len, const double *src, double *dst);
```

接口说明: 计算输入向量的双曲正切值, 输出为向量。

$$dst_i = \tanh(src_i); \quad 0 \leq i < len$$

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	复数单(双)精度浮点类型	输入向量 src	输入
dst	复数单(双)精度浮点类型	输出向量 dst	输出

3.3.2.4 Asinh

函数定义

```
void vasinh_f(const int len, const float *src, float *dst);
```

```
void vasinh_d(const int len, const double *src, double *dst);
```

接口说明: 计算输入向量的反双曲正弦值, 输出为向量。

$$dst_i = \sinh(src_i); \quad 0 \leq i < len$$

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.2.5 Acosh

函数定义

```
void vacosh_f(const int len, const float *src, float *dst);
```

```
void vacosh_d(const int len, const double *src, double *dst);
```

接口说明: 计算输入向量的反双曲余弦值, 输出为向量。

$$dst_i = acosh(src_i); \quad 0 \leq i < len$$

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.2.6 Atanh

函数定义

```
void vatanh_f(const int len, const float *src, float *dst);
void vatanh_d(const int len, const double *src, double *dst);
```

接口说明: 计算输入向量的双曲正切值, 输出为向量。

$$dst_i = atanh(src_i); \quad 0 \leq i < len$$

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3 指数对数函数

3.3.3.1 Exp

函数定义

```
void vexp_f(const int len, const float *src, float *dst);
void vexp_d(const int len, const double *src, double *dst);
void cvexp_f(const int len, const float complex *src, float complex *dst);
void cvexp_d(const int len, const double complex *src, double complex *dst);
```

接口说明: 计算输入以自然对数为底的指数, 输入输出均为向量。

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点类型	输入向量 src	输入
dst	(复数)单(双)精度浮点类型	输出向量 dst	输出

3.3.3.2 Exp2

函数定义

```
void vexp2_f(const int len, const float *src, float *dst);  
void vexp2_d(const int len, const double *src, double *dst);
```

接口说明：计算输入以 2 为底的指数，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.3 Exp10

函数定义

```
void vexp10_f(const int len, const float *src, float *dst);  
void vexp10_d(const int len, const double *src, double *dst);
```

接口说明：计算输入以 10 为底的指数，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.4 Expm1

函数定义

```
void vexpm1_f(const int len, const float *src, float *dst);  
void vexpm1_d(const int len, const double *src, double *dst);
```

接口说明：计算输入以自然对数 e 为底的指数减 1，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.5 Ln

函数定义

```
void vln_f(const int len, const float *src, float *dst);  
void vln_d(const int len, const double *src, double *dst);  
void cvln_f(const int len, const float complex *src, float complex *dst);  
void cvln_d(const int len, const double complex *src, double complex *dst);
```

接口说明：计算输入以自然对数为底的对数，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点类型	输入向量 src	输入
dst	(复数)单(双)精度浮点类型	输出向量 dst	输出

3.3.3.6 Log2

函数定义

```
void vlog2_f(const int len, const float *src, float *dst);  
void vlog2_d(const int len, const double *src, double *dst);
```

接口说明：计算输入以 2 为底的对数，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.7 Log10

函数定义

```
void vlog10_f(const int len, const float *src, float *dst);  
void vlog10_d(const int len, const double *src, double *dst);
```

接口说明：计算输入以 10 为底的对数，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.8 Log1p

函数定义

```
void vlog1p_f(const int len, const float *src, float *dst);  
void vlog1p_d(const int len, const double *src, double *dst);
```

接口说明：计算输入以自然数为底，输入值加 1 的对数值，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.9 Pow

函数定义

```
void vpow_f(const int len, const float *src1, const float *src2, float *dst);  
void vpow_d(const int len, const double *src1, const double *src2, double *dst);
```

接口说明：计算 src1 的 src2 次幂，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

函数定义

```
void svpow_d(const int len, const double src1, const double *src2, double *dst);  
void svpow_f(const int len, const float src1, const float *src2, float *dst);
```

接口说明：计算输入 src2 的 src1 次幂的值，src2 为向量，src1 为标量，输出为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入标量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.10 Logb

函数定义

```
void vlogb_f(const int len, const float *src, float *dst);  
void vlogb_d(const int len, const double *src, double *dst);
```

接口说明：计算输入 src 的无偏差指数值，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.11 llogb

函数定义

```
void vilogb_f(const int len, const float *src, int *dst);  
void vilogb_d(const int len, const double *src, int *dst);
```

接口说明：计算输入向量 src 的无偏差指数值。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	整型	输出向量 dst	输出

3.3.3.12 Sqr

函数定义

```
void vsqr_f(const int len, const float *src, float *dst);  
void vsqr_d(const int len, const double *src, double *dst);
```

接口说明：计算输入的平方，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.13 Sqrt

函数定义

```
void vsqrt_f(const int len, const float *src, float *dst);  
void vsqrt_d(const int len, const double *src, double *dst);  
void cvsqrt_f(const int len, const float complex *src, float complex *dst);  
void cvsqrt_d(const int len, const double complex *src, double complex *dst);
```

接口说明：计算输入的平方根，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点类型	输入向量 src	输入
dst	(复数)单(双)精度浮点类型	输出向量 dst	输出

函数定义

```
void vrsqrt_d(const int len, const double *src, double *dst);  
void vrsqrt_f(const int len, const float *src, float *dst);
```

接口说明：计算输入的平方根的倒数，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.14 Cbrt

函数定义

```
void vcbrt_f(const int len, const float *src, float *dst);  
void vcbrt_d(const int len, const double *src, double *dst);
```

接口说明：计算输入的立方根，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.3.15 Hypot

函数定义

```
void vhypot_f(const int len, const float *src1, const float *src2, float *dst);  
void vhypot_d(const int len, const double *src1, const double *src2, double *dst);
```

接口说明：求输入平方和的平方根，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输出向量 src2	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.4 特殊函数和随机函数

3.3.4.1 Erf

函数定义

```
void verf_f(const int len, const float *src, float *dst);  
void verf_d(const int len, const double *src, double *dst);
```

接口说明：计算输入的误差函数，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.4.2 Erfc

函数定义

```
void verfc_f(const int len, const float *src, float *dst);  
void verfc_d(const int len, const double *src, double *dst);
```

接口说明：计算输入的补余误差函数，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
----	----	----	-------

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.4.3 Tgamma

函数定义

```
void vtgamma_f(const int len, const float *src, float *dst);
```

```
void vtgamma_d(const int len, const double *src, double *dst);
```

接口说明：计算输入元素的伽马函数，输入输出均为向量；

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.4.4 Lgamma

函数定义

```
void vlgamma_d(const int len, const double *src, double *dst);
```

```
void vlgamma_f(const int len, const float *src, float *dst);
```

接口说明：计算输入元素的伽马函数绝对值的自然对数，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.4.5 Rand

函数定义

```
vml_randstate *randcreate(unsigned long int seed, unsigned long int numseqs,  
unsigned long int id, vml_rng portable);
```

```
int randdestroy(vml_randstate *state);
```

函数说明：随机数支撑函数，randcreate 为生成结构体函数，返回 vml_randstate 类型指针，randcreate 为释放结构体函数，输入为 vml_randstate 类型指针，返回为整型。

randcreate 函数参数说明

参数	类型	描述	输入/输出
seed	无符号长整型	随机数生成器种子	输入
numseqs	无符号长整型	处理器数	输入
id	无符号长整型	处理器 ID	输入
portable	vml_rng 型		输入

函数定义

```
void vrandu_f(const int len, vml_randstate *state, float *dst);
void vrandu_d(const int len, vml_randstate *state, double *dst);
void vrandn_f(const int len, vml_randstate *state, float *dst);
void vrandn_d(const int len, vml_randstate *state, double *dst);
void cvrandn_f(const int len, vml_randstate *state, float _Complex *dst);
void cvrandn_d(const int len, vml_randstate *state, double _Complex *dst);
void cvrandu_f(const int len, vml_randstate *state, float _Complex *dst);
void cvrandu_d(const int len, vml_randstate *state, double _Complex *dst);
```

函数说明：根据输入结构体指针产生随机数向量，(c)vrandn_d(f)为生成正态分布的随机向量，(c)vrandu_d(f)为生成均匀分布的随机向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
state	vml_randstate 型	输入指针 state	输入
dst	(复数)单(双)精度浮点类型	输出向量 dst	输出

3.3.5 比较函数和最值函数

3.3.5.1 Llt

函数定义

```
void vllt_d(const int len, const double *src1, const double *src2, unsigned int *dst);
void vllt_f(const int len, const float *src1, const float *src2, unsigned int *dst);
```

函数说明：比较输入 src1 和 src2，src1 小于 src2 的对应位置的 dst 值为 1，其余位置值为 0。

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	无符号整型	输出向量 dst	输出

3.3.5.2 Lle

函数定义

```
void vlle_d(const int len, const double *src1, const double *src2, unsigned int *dst);
```

```
void vlle_f(const int len, const float *src1, const float *src2, unsigned int *dst);
```

函数说明: 比较输入 src1 和 src2, src1 小于等于 src2 的对应位置的 dst 值为 1, 其余位置值为 0。

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	无符号整型	输出向量 dst	输出

3.3.5.3 Lgt

函数定义

```
void vlgt_d(const int len, const double *src1, const double *src2, unsigned int *dst);
```

```
void vlgt_f(const int len, const float *src1, const float *src2, unsigned int *dst);
```

函数说明: 比较输入 src1 和 src2, src1 大于 src2 的对应位置的 dst 值为 1, 其余位置值为 0。

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	无符号整型	输出向量 dst	输出

3.3.5.4 Lge

函数定义

```
void vlge_d(const int len, const double *src1, const double *src2, unsigned int *dst);
```

```
void vlge_f(const int len, const float *src1, const float *src2, unsigned int *dst);
```

函数说明：比较输入 src1 和 src2，src1 大于等于 src2 的对应位置的 dst 值为 1，其余位置值为 0。

参数说明：

参数	类型	描述	输入/输出
Len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
Dst	无符号整型	输出向量 dst	输出

3.3.5.5 Leq

函数定义

```
void vleq_d(const int len, const double *src1, const double *src2, unsigned int *dst);
```

```
void vleq_f(const int len, const float *src1, const float *src2, unsigned int *dst);
```

函数说明：比较输入 src1 和 src2，src1 等于 src2 的对应位置的 dst 值为 1，其余位置值为 0。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	无符号整型	输出向量 dst	输出

3.3.5.6 Lne

函数定义

```
void vlne_d(const int len, const double *src1, const double *src2, unsigned int *dst);
```

```
void vlne_f(const int len, const float *src1, const float *src2, unsigned int *dst);
```

函数说明：比较输入 src1 和 src2，src1 不等于 src2 的对应位置的 dst 值为 1，其余位置值为 0。

参数说明：

参数	类型	描述	输入/输出
----	----	----	-------

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	无符号整型	输出向量 dst	输出

3.3.5.7 整型比较函数

函数定义

```
void vllt_i(const int len, const int *src1, const int *src2, unsigned int *dst);
void vlle_i(const int len, const int *src1, const int *src2, unsigned int *dst);
void vlgt_i(const int len, const int *src1, const int *src2, unsigned int *dst);
void vlge_i(const int len, const int *src1, const int *src2, unsigned int *dst);
void vleq_i(const int len, const int *src1, const int *src2, unsigned int *dst);
void vlne_i(const int len, const int *src1, const int *src2, unsigned int *dst);
```

函数说明：整型向量比较函数，输入为 src1 和 src2，src1 小于(小于等于/大于/大于等于/等于/不等于)src2 对应位置的 dst 值为 1，其余位置值为 0。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	整型	输入向量 src1	输入
src2	整型	输入向量 src2	输入
dst	无符号整型	输出向量 dst	输出

3.3.5.8 Min & Max

函数定义

```
void vmin_d(const int len, const double *src1, const double *src2, double *dst);
void vmin_f(const int len, const float *src1, const float *src2, float *dst);
void vmax_d(const int len, const double *src1, const double *src2, double *dst);
void vmax_f(const int len, const float *src1, const float *src2, float *dst);
```

函数说明：输出向量 src1，src2 中较小(较大)的值，存储在向量 dst 中。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.5.9 Minmg &Maxmg

函数定义

```
void vminmg_d(const int len, const double *src1, const double *src2, double *dst);  
void vminmg_f(const int len, const float *src1, const float *src2, float *dst);  
void vmaxmg_d(const int len, const double *src1, const double *src2, double *dst);  
void vmaxmg_f(const int len, const float *src1, const float *src2, float *dst);
```

函数说明：求输入向量 src1，src2 的绝对值并输出较小(较大)的绝对值，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量 src1	输入
src2	单(双)精度浮点类型	输入向量 src2	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

函数定义

```
void vminmgsq_d(const int len, const double _Complex *src1, const double  
_Complex *src2, double *dst);  
void vminmgsq_f(const int len, const float _Complex *src1, const float _Complex  
*src2, float *dst);  
void vmaxmgsq_d(const int len, const double _Complex *src1, const double  
_Complex *src2, double *dst);  
void vmaxmgsq_f(const int len, const float _Complex *src1, const float _Complex  
*src2, float *dst);
```

函数说明：比较输入 src1，src2 的模的平方，输出较小(较大)的模的平方。输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	复数单(双)精度浮点类型	输入向量 src1	输入
src2	复数单(双)精度浮点类型	输入向量 src2	输入
dst	单(双)精度浮点类型	输出向量 dst	输出

3.3.5.10 Minval & Maxval

函数定义

```
double vminval_d(const int len, const double *src, unsigned int *indx);  
float vminval_f(const int len, const float *src, unsigned int *indx);  
double vmaxval_d(const int len, const double *src, unsigned int *indx);  
float vmaxval_f(const int len, const float *src, unsigned int *indx);
```

函数说明：返回输入向量的最小(最大)值并将该值所在地址存储在 indx 所指向位置。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
Indx	无符号整型	输出所寻找值的地址	输出
返回	单(双)精度浮点类型	返回值	输出

3.3.5.11 Minmgval & Maxmgval

函数定义

```
double vminmgval_d(const int len, const double *src, unsigned int *indx);  
float vminmgval_f(const int len, const float *src, unsigned int *indx);  
double vmaxmgval_d(const int len, const double *src, unsigned int *indx);  
float vmaxmgval_f(const int len, const float *src, unsigned int *indx);
```

函数说明：返回输入向量中绝对值最小(最大)的值的绝对值并存储该值的地址。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量 src	输入
Indx	无符号整型	输出所寻找值的地址	输出
返回	单(双)精度浮点类型	返回值	输出

3.3.6 基本运算函数

3.3.6.1 Add

函数定义

```
void vadd_f(const int len, const float *src1, const float *src2, float *dst);  
void vadd_d(const int len, const double *src1, const double *src2, double *dst);
```



```

void cvadd_f(const int len, const float complex *src1, const float complex *src2,
float complex *dst);
void cvadd_d(const int len, const double complex *src1, const double complex
*src2, double complex *dst);
void svadd_d(const int len, const double src1, const double *src2, double *dst);
void svadd_f(const int len, const float src1, const float *src2, float *dst);
void csvadd_d(const int len, const double _Complex src1, const double _Complex
*src2, double _Complex *dst);
void csvadd_f(const int len, const float _Complex src1, const float _Complex *src2,
float _Complex *dst);
void rcvadd_d(const int len, const double *src1, const double _Complex *src2,
double _Complex *dst);
void rcvadd_f(const int len, const float *src1, const float _Complex *src2, float
_Complex *dst);
void rscvadd_d(const int len, const double src1, const double _Complex *src2,
double _Complex *dst);
void rscvadd_f(const int len, const float src1, const float _Complex *src2, float
_Complex *dst);
void svadd_i(const int len, const int src1, const int *src2, int *dst);
void vadd_i(const int len, const int *src1, const int *src2, int *dst);

```

函数说明：加法运算。

函数名	说明
vadd_f(_d)	单(双)精度向量加
cvadd_f(_d)	复数单(双)精度向量加
svadd_f(_d)	单(双)精度常量加向量
csvadd_f(_d)	复数单(双)精度常量加向量
rcvadd_f(_d)	单(双)精度实数向量加复数向量
rscvadd_f(_d)	单(双)精度实数常量加复数向量
svadd_i	整型常量加向量
vadd_i	整型向量加向量

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	(复数)单(双)精度浮点类型/整型	输入向量/常量	输入
src2	(复数)单(双)精度浮点类型/整型	输入向量	输入
dst	(复数)单(双)精度浮点类型/整型	输出向量	输出

3.3.6.2 Sub

函数定义

```
void vsub_f(const int len, const float *src1, const float *src2, float *dst);
void vsub_d(const int len, const double *src1, const double *src2, double *dst);
void cvsub_f(const int len, const float complex *src1, const float complex *src2,
float complex *dst);
void cvsub_d(const int len, const double complex *src1, const double complex
*src2, double complex *dst);
void svsub_d(const int len, const double src1, const double *src2, double *dst);
void svsub_f(const int len, const float src1, const float *src2, float *dst);
void vssub_d(const int len, const double *src1, const double src2, double *dst);
void vssub_f(const int len, const float *src1, const float src2, float *dst);
void csvsub_d(const int len, const double _Complex src1, const double _Complex
*src2, double _Complex *dst);
void csvsub_f(const int len, const float _Complex src1, const float _Complex *src2,
float _Complex *dst);
void rcvsub_d(const int len, const double *src1, const double _Complex *src2,
double _Complex *dst);
void rcvsub_f(const int len, const float *src1, const float _Complex *src2, float
_Complex *dst);
void rscvsub_d(const int len, const double src1, const double _Complex *src2,
double _Complex *dst);
void rscvsub_f(const int len, const float src1, const float _Complex *src2, float
_Complex *dst);
void crvsub_f(const int len, const float _Complex *src1, const float *src2, float
_Complex *dst);
void crvsub_d(const int len, const double _Complex *src1, const double *src2,
double _Complex *dst);
void svsub_i(const int len, const int src1, const int *src2, int *dst);
void vsub_i(const int len, const int *src1, const int *src2, int *dst);
```

函数说明：减法运算。

函数名	说明
vsub_f(_d)	单(双)精度向量减
cvsub_f(_d)	复数单(双)精度向量减
svsub_f(_d)	单(双)精度常量减向量
vssub_f(_d)	单(双)精度向量减常量
csvsub_f(_d)	复数单(双)精度常量减向量

函数名	说明
rcvsub_f(_d)	单(双)精度实数向量减复数向量
rscvsub_f(_d)	单(双)精度实数常量减复数向量
crvsub_f(_d)	单(双)精度复数向量减实数向量
svsub_i	整型常量减向量
vsub_i	整型向量减向量

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	(复数)单(双)精度浮点类型/整型	输入向量/常量	输入
src2	(复数)单(双)精度浮点类型/整型	输入向量/常量	输入
dst	(复数)单(双)精度浮点类型/整型	输出向量	输出

3.3.6.3 Mul

函数定义

```

void vmul_f(const int len, const float *src1, const float *src2, float *dst);
void vmul_d(const int len, const double *src1, const double *src2, double *dst);
void cvmul_f(const int len, const float complex *src1, const float complex *src2,
float complex *dst);
void cvmul_d(const int len, const double complex *src1, const double complex
*src2, double complex *dst);
void csvmul_d(const int len, const double _Complex src1, const double _Complex
*src2, double _Complex *dst);
void csvmul_f(const int len, const float _Complex src1, const float _Complex *src2,
float _Complex *dst);
void rcvmul_d(const int len, const double *src1, const double _Complex *src2,
double _Complex *dst);
void rcvmul_f(const int len, const float *src1, const float _Complex *src2, float
_Complex *dst);
void rscvmul_d(const int len, const double src1, const double _Complex *src2,
double _Complex *dst);
void rscvmul_f(const int len, const float src1, const float _Complex *src2, float
_Complex *dst);
void svmul_d(const int len, const double src1, const double *src2, double *dst);
void svmul_f(const int len, const float src1, const float *src2, float *dst);
void cvjmul_d(const int len, const double _Complex *src1, const double _Complex
*src2, double _Complex *dst);

```

```
void cvjmul_f(const int len, const float _Complex *src1, const float _Complex *src2,
float _Complex *dst);
void vmul_i(const int len, const int *src1, const int *src2, int *dst);
void svmul_i(const int len, const int src1, const int *src2, int *dst);
```

函数说明：乘法运算。

函数名	说明
vmul_f(_d)	单(双)精度向量乘
cvmul_f(_d)	复数单(双)精度向量乘
csvmul_f(_d)	复数单(双)精度常量乘向量
rcvmul_f(_d)	单(双)精度实数向量乘复数向量
rscvmul_f(_d)	单(双)精度实数常量乘复数向量
svmul_f(_d)	单(双)精度常量乘向量
cvjmul_f(_d)	复数单(双)精度向量乘向量的共轭
vmul_i	整型向量乘向量
svmul_i	整型常量乘向量

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	(复数)单(双)精度浮点类型/整型	输入向量/常量	输入
src2	(复数)单(双)精度浮点类型/整型	输入向量	输入
dst	(复数)单(双)精度浮点类型/整型	输出向量	输出

3.3.6.4 Div

函数定义

```
void vdiv_f(const int len, const float *src1, const float *src2, float *dst);
void vdiv_d(const int len, const double *src1, const double *src2, double *dst);
void cvdiv_f(const int len, const float _Complex *src1, const float _Complex *src2,
float _Complex *dst);
void cvdiv_d(const int len, const double _Complex *src1, const double _Complex
*src2, double _Complex *dst);
void crvdiv_f(const int len, const float _Complex *src1, const float *src2, float
_Complex *dst);
void crvdiv_d(const int len, const double _Complex *src1, const double *src2,
double _Complex *dst);
void csvdiv_f(const int len, const float _Complex src1, const float _Complex *src2,
float _Complex *dst);
void csvdiv_d(const int len, const double _Complex src1, const double _Complex
```

```

*src2, double _Complex *dst );
    void cvrsdiv_d(const int len, const double _Complex *src1, const double src2,
double _Complex *dst );
    void cvrsdiv_f(const int len, const float _Complex *src1, const float src2, float
_Complex *dst );
    void rcvdiv_d(const int len, const double *src1, const double _Complex *src2,
double _Complex *dst);
    void rcvdiv_f(const int len, const float *src1, const float _Complex *src2, float
_Complex *dst);
    void svdiv_d(const int len, const double src1, const double *src2, double *dst);
    void svdiv_f(const int len, const float src1, const float *src2, float *dst);
    void rscvdiv_d(const int len, const double src1, const double _Complex *src2,
double _Complex *dst);
    void rscvdiv_f(const int len, const float src1, const float _Complex *src2, float
_Complex *dst);
    void vsdiv_d(const int len, const double *src1, const double src2, double *dst);
    void vsdiv_f(const int len, const float *src1, const float src2, float *dst);

```

函数说明：除法运算。

函数名	说明
vdiv_f(_d)	单(双)精度向量除以向量
cvdiv_f(_d)	复数单(双)精度向量除以向量
crvdiv_f(_d)	单(双)精度复数向量除以实数向量
csvdiv_f(_d)	单(双)精度复数向量除以复数常量
cvrsdiv_f(_d)	单(双)精度复数向量除以实数常量
rcvdiv_f(_d)	单(双)精度实数向量除以复数向量
svdiv_f(_d)	单(双)精度常量除以向量
rscvdiv_f(_d)	单(双)精度实数常量除以复数向量
vsdiv_f(_d)	单(双)精度向量除以常量

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	(复数)单(双)精度浮点类型	输入向量/常量	输入
src2	(复数)单(双)精度浮点类型	输入向量/常量	输入
dst	(复数)单(双)精度浮点类型	输出向量	输出

3.3.6.5 Ma & Am & Msb & Sbm

函数定义

```
void vam_d(const int len, const double *src1, const double *src2, const double
*src3, double *dst);
void vam_f(const int len, const float *src1, const float *src2, const float *src3, float
*dst);
void vma_d(const int len, const double *src1, const double *src2, const double
*src3, double *dst);
void vma_f(const int len, const float *src1, const float *src2, const float *src3, float
*dst);
void vsam_d(const int len, const double *src1, const double src2, const double
*src3, double *dst);
void vsam_f(const int len, const float *src1, const float src2, const float *src3, float
*dst);
void vsma_d(const int len, const double *src1, const double src2, const double
*src3, double *dst);
void vsma_f(const int len, const float *src1, const float src2, const float *src3, float
*dst);
void vmsa_d(const int len, const double *src1, const double *src2, const double
src3, double *dst);
void vmsa_f(const int len, const float *src1, const float *src2, const float src3, float
*dst);
void vmsma_d(const int len, const double *src1, const double src2, const double
src3, double *dst);
void vmsma_f(const int len, const float *src1, const float src2, const float src3,
float *dst);
void cvam_d(const int len, const double _Complex *src1, const double _Complex
*src2, const double _Complex *src3, double _Complex *dst);
void cvam_f(const int len, const float _Complex *src1, const float _Complex *src2,
const float _Complex *src3, float _Complex *dst);
void cvma_d(const int len, const double _Complex *src1, const double _Complex
*src2, const double _Complex *src3, double _Complex *dst);
void cvma_f(const int len, const float _Complex *src1, const float _Complex *src2,
const float _Complex *src3, float _Complex *dst);
void cvsam_d(const int len, const double _Complex *src1, const double _Complex
src2, const double _Complex *src3, double _Complex *dst);
void cvsam_f(const int len, const float _Complex *src1, const float _Complex src2,
const float _Complex *src3, float _Complex *dst);
void cvsma_d(const int len, const double _Complex *src1, const double _Complex
src2, const double _Complex *src3, double _Complex *dst);
```

```

void cvsma_f(const int len, const float _Complex *src1, const float _Complex src2,
const float _Complex *src3, float _Complex *dst);
void cvmsa_d(const int len, const double _Complex *src1, const double _Complex
*src2, const double _Complex src3, double _Complex *dst);
void cvmsa_f(const int len, const float _Complex *src1, const float _Complex *src2,
const float _Complex src3, float _Complex *dst);
void cvsmsa_d(const int len, const double _Complex *src1, const double
_Complex src2, const double _Complex src3, double _Complex *dst);
void cvsmsa_f(const int len, const float _Complex *src1, const float _Complex src2,
const float _Complex src3, float _Complex *dst);
void vsbm_d(const int len, const double *src1, const double *src2, const double
*src3, double *dst);
void vsbm_f(const int len, const float *src1, const float *src2, const float *src3, float
*dst);
void vmsb_d(const int len, const double *src1, const double *src2, const double
*src3, double *dst);
void vmsb_f(const int len, const float *src1, const float *src2, const float *src3, float
*dst);
void cvsbm_d(const int len, const double _Complex *src1, const double _Complex
*src2, const double _Complex *src3, double _Complex *dst);
void cvsbm_f(const int len, const float _Complex *src1, const float _Complex *src2,
const float _Complex *src3, float _Complex *dst);
void cvmsb_d(const int len, const double _Complex *src1, const double _Complex
*src2, const double _Complex *src3, double _Complex *dst);
void cvmsb_f(const int len, const float _Complex *src1, const float _Complex *src2,
const float _Complex *src3, float _Complex *dst);

```

函数说明：乘加，乘减，加乘，减乘运算。

函数名	说明
vam_f(_d)	单(双)精度向量加向量的和乘向量
vma_f(_d)	单(双)精度向量乘向量的积加向量
vsam_f(_d)	单(双)精度向量加常量的和乘向量
vsma_f(_d)	单(双)精度向量乘常量的积加向量
vmsa_f(_d)	单(双)精度向量乘向量的积加常量
vsmsa_f(_d)	单(双)精度向量乘常量的积加常量
cvam_f(_d)	复数单(双)精度向量加向量的和乘向量
cvma_f(_d)	复数单(双)精度向量乘向量的积加向量
cvsam_f(_d)	复数单(双)精度向量加常量的和乘向量
cvsma_f(_d)	复数单(双)精度向量乘常量的积加向量

函数名	说明
cvmsa_f(_d)	复数单(双)精度向量乘向量的积加常量
cvmsa_d_f(_d)	复数单(双)精度向量乘常量的积加常量
vsbm_f(_d)	单(双)精度向量减向量的差乘向量
vmsb_f(_d)	单(双)精度向量乘向量的积减向量
cvsbm_f(_d)	复数单(双)精度向量减向量的差乘向量
cvmsb_f(_d)	复数单(双)精度向量乘向量的积减向量

参数说明

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	(复数)单(双)精度浮点类型	输入向量	输入
src2	(复数)单(双)精度浮点类型	输入向量/常量	输入
src3	(复数)单(双)精度浮点类型	输入向量/常量	输入
dst	(复数)单(双)精度浮点类型	输出向量	输出

3.3.6.6 mag

函数定义

```
void vmag_f(const int len, const float *src, float *dst);
void vmag_d(const int len, const double *src, double *dst);
void cvmag_f(const int len, const float complex *src, float *dst);
void cvmag_d(const int len, const double complex *src, double *dst);
```

函数说明：计算输入的绝对值，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

函数定义

```
void vmag_i(const int len, const int *src, int *dst);
```

函数说明：取输入每个元素的绝对值。

参数说明

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	整型	输入向量	输入

参数	类型	描述	输入/输出
dst	整型	输出向量	输出

3.3.6.7 magsq

函数定义

```
void vcmagsq_d(const int len, const double _Complex *src, double *dst);
```

```
void vcmagsq_f(const int len, const float _Complex *src, float *dst);
```

函数说明：计算输入复数模的平方，输入输出均为向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	复数单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.8 arg

函数定义

```
void varg_f(const int len, const float complex *src, float *dst);
```

```
void varg_d(const int len, const double complex *src, double *dst);
```

函数说明：计算输入复数向量每个元素在区间 $[-\pi, \pi]$ 上的弧度。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	复数单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.9 conj

函数定义

```
void cvconj_f(const int len, const float complex *src, float complex *dst);
```

```
void cvconj_d(const int len, const double complex *src, double complex *dst);
```

函数说明：计算复数向量每个元素的共轭值。

参数说明：

参数	类型	描述	输入/输出
----	----	----	-------

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	复数单(双)精度浮点类型	输入向量	输入
dst	复数单(双)精度浮点类型	输出向量	输出

3.3.6.10 fmod

函数定义

```
void vfmod_f(const int len, const float *src1, const float *src2, float *dst);
```

```
void vfmod_d(const int len, const double *src1, const double *src2, double *dst);
```

函数说明：计算向量 src1，src2 每个对应元素商的浮点余数。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量	输入
src2	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.11 inv

函数定义

```
void vinv_f(const int len, const float *src, float *dst);
```

```
void vinv_d(const int len, const double *src, double *dst);
```

函数说明：计算输入每个元素的倒数。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.12 rint

函数定义

```
void vrint_f(const int len, const float *src, float *dst);
```

```
void vrint_d(const int len, const double *src, double *dst);
```

函数说明：将输入向量的每个元素四舍五入为浮点格式的整数。

参数说明

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.13 trunc

函数定义

```
void vtrunc_f(const int len, const float *src, float *dst);
```

```
void vtrunc_d(const int len, const double *src, double *dst);
```

函数说明：将输入向量的每个元素舍入为不大于该元素的最大浮点格式的整数。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.14 floor

函数定义

```
void vfloor_f(const int len, const float *src, float *dst);
```

```
void vfloor_d(const int len, const double *src, double *dst);
```

函数说明：将输入向量的每个元素舍入为不大于该元素的最大浮点格式的整数。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.15 round

函数定义

```
void vround_f(const int len, const float *src, float *dst);
```

```
void vround_d(const int len, const double *src, double *dst);
```

函数说明：将输入向量的每个元素四舍五入为浮点格式的整数。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.16 ceil

函数定义

```
void vceil_f(const int len, const float *src, float *dst);
```

```
void vceil_d(const int len, const double *src, double *dst);
```

函数说明：将输入向量的每个元素舍入为不小于该元素的最大浮点格式的整数。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.17 fdim

函数定义

```
void vfdim_f(const int len, const float *src1, const float *src2, float *dst);
```

```
void vfdim_d(const int len, const double *src1, const double *src2, double *dst);
```

函数说明：计算输入 src1 与 src2 之间每个元素的正数差。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量	输入
src2	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.6.18 neg

函数定义

```
void vneg_d(const int len, const double *src, double *dst);  
void vneg_f(const int len, const float *src, float *dst);  
void cvneg_d(const int len, const double _Complex *src, double _Complex *dst);  
void cvneg_f(const int len, const float _Complex *src, float _Complex *dst);
```

函数说明：计算输入每个元素乘-1 的值。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点类型	输入向量	输入
dst	(复数)单(双)精度浮点类型	输出向量	输出

函数定义

```
void vneg_i(const int len, const int *src, int *dst);
```

函数说明：计算每个输入元素乘-1 的值。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	整型	输入向量	输入
dst	整型	输出向量	输出

3.3.6.19 not

函数定义

```
void vnot_i(const int len, const int *src, int *dst);
```

函数说明：将输入的每个元素按位取反。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	整型	输入向量	输入
dst	整型	输出向量	输出

3.3.6.20 and&xor&or

函数定义

```
void vand_i(const int len, const int *src1, const int *src2, int *dst);
void vxor_i(const int len, const int *src1, const int *src2, int *dst);
void vor_i(const int len, const int *src1, const int *src2, int *dst);
```

函数说明: vand_i, 计算输入 src1 与 src2 的与; vxor_i, 计算输入 src1 与 src2 的异或; vor_i, 计算输入 src1 与 src2 的或。

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	整型	输入向量	输入
src2	整型	输入向量	输入
dst	整型	输出向量	输出

3.3.7 其它函数

3.3.7.1 Fill

函数定义

```
void vfill_d(const int len, const double src, double *dst);
void vfill_f(const int len, const float src, float *dst);
void cvfill_d(const int len, const double _Complex src, double _Complex *dst);
void cvfill_f(const int len, const float _Complex src, float _Complex *dst);
```

函数说明: 用所给输入常量 src 填充向量 dst。

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点数	输入常量	输入
dst	(复数)单(双)精度浮点类型	输出向量 dst	输出

3.3.7.2 Real & Imag

函数定义

```
void cvreal_d(const int len, const double _Complex *src, double *dst);
void cvreal_f(const int len, const float _Complex *src, float *dst);
```

```
void cvimag_d(const int len, const double _Complex *src, double *dst);
```

```
void cvimag_f(const int len, const float _Complex *src, float *dst);
```

函数说明：计算输入复数向量的实部或虚部。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	复数单(双)精度浮点数	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.7.3 Cmplx

函数定义

```
void vcmlx_d(const int len, const double *src1, const double *src2, double  
_Complex *dst);
```

```
void vcmlx_f(const int len, const float *src1, const float *src2, float _Complex  
*dst);
```

函数说明：输出以 src1 为实部分 src2 为虚部的复数向量。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点数	输入向量	输入
src2	单(双)精度浮点数	输入向量	输入
dst	复数单(双)精度浮点类型	输出向量	输出

3.3.7.4 Swap

函数定义

```
void vswap_d(const int len, const double *src1, const double *src2);
```

```
void vswap_f(const int len, const float *src1, const float *src2);
```

```
void cvswap_d(const int len, const double _Complex *src1, const double  
_Complex *src2);
```

```
void cvswap_f(const int len, const float _Complex *src1, const float _Complex  
*src2);
```

函数说明：交换向量 src1，src2。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	(复数)单(双)精度浮点数	输入/输出向量	输入/输出

参数	类型	描述	输入/输出
src2	(复数)单(双)精度浮点数	输入/输出向量	输入/输出

3.3.7.5 Frexp

函数定义

```
void vfrexp_f(const int len, const float *src, int *dst1, float *dst2);  
void vfrexp_d(const int len, const double *src, int *dst1, double *dst2);
```

函数说明：将输入的每个元素分解为归一化分数和 2 的整数幂。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点数	输入向量	输入
dst1	整型数	输出向量	输出
dst2	单(双)精度浮点数	输出向量	输出

3.3.7.6 Ldexp

函数定义

```
void vldexp_f(const int len, const float *src1, const int *src2, float *dst);  
void vldexp_d(const int len, const double *src1, const int *src2, double *dst);
```

函数说明：计算 src1 中元素乘以 2 的 src2 中对应元素次幂的积。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量	输入
src2	整型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.7.7 Modf

函数定义

```
void vmodf_f(const int len, const float *src, const float *dst1, float *dst2);  
void vmodf_d(const int len, const double *src, const double *dst1, double *dst2);
```

函数说明：将输入的每个元素分解为浮点格式具有相同符号的整数和小数，整数部分存储在 dst1 中，小数部分存储在 dst2 中。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst1	单(双)精度浮点类型	输出向量	输出
dst2	单(双)精度浮点类型	输出向量	输出

3.3.7.8 Scalbn

函数定义

```
void vscalbn_f(const int len, const float *src1, const int *src2, float *dst);
void vscalbn_d(const int len, const double *src1, const int *src2, double *dst);
```

函数说明：计算 src1 中元素乘以 2 的 src2 中对应元素次幂的积。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量	输入
src2	整型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.7.9 Remainder

函数定义

```
void vremainder_f(const int len, const float *src1, const float *src2, float *dst);
void vremainder_d(const int len, const double *src1, const double *src2, double *dst);
```

函数说明：计算输入 src1 与 src2 对应元素的商的余数。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量	输入
src2	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.7.10 Nearbyint

函数定义

```
void vnearbyint_f(const int len, const float *src, float *dst);
```

```
void vnearbyint_d(const int len, const double *src, double *dst);
```

函数说明：将输入的元素四舍五入为浮点格式的整数。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.7.11 Remquo

函数定义

```
void vremquo_f(const int len, const float *src1, const float *src2, const int *dst1, float *dst2);
```

```
void vremquo_d(const int len, const double *src1, const double *src2, const int *dst1, double *dst2);
```

函数说明：计算输入 src1 与 src2 对应每个元素商的余数，将商的符号和近似值存储在 dst1 中，将余数存储在 dst2 中。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量, src1 中元素做分子	输入
src2	单(双)精度浮点类型	输入向量, src2 中元素做分母	输入
dst1	整型	输出向量	输出
dst2	单(双)精度浮点类型	输出向量	输出

3.3.7.12 Copysign

函数定义

```
void vcopysign_f(const int len, const float *src1, const float *src2, float *dst);
```

```
void vcopysign_d(const int len, const double *src1, const double *src2, double *dst);
```

函数说明：输出和 src1 中元素具有相同数值，和 src2 中元素具有相同符号的值。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量, dst 与 src1 具有相同数值	输入
src2	单(双)精度浮点类型	输入向量, dst 与 src2 具有相同符号	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.7.13 Nextafter

函数定义

```
void vnextafter_d(const int len, const double *src1, const double *src2, double *dst);
```

```
void vnextafter_f(const int len, const float *src1, const float *src2, float *dst);
```

函数说明：输出从 src1 中元素开始向 src2 中对应元素方向的下一个可以表示的浮点值。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量	输入
src2	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.7.14 Nexttoward

函数定义

```
void vnexttoward_d(const int len, const double *src1, const long double *src2, double *dst);
```

```
void vnexttoward_f(const int len, const float *src1, const long double *src2, float *dst);
```

函数说明：输出从 src1 中元素开始向 src2 中对应元素方向的下一个可以表示的浮点值。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量	输入
src2	长双精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

3.3.7.15 Polar

函数定义

```
void cvpolar_f(const int len, const float _Complex *src, float *dst1, float *dst2);
```

```
void cvpolar_d(const int len, const double _Complex *src, double *dst1, double *dst2);
```

函数说明：将输入的复数元素从直角坐标形式转换为极坐标形式。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	复数单(双)精度浮点类型	输入向量	输入
dst1	单(双)精度浮点类型	输出向量，存储半径	输出
dst2	单(双)精度浮点类型	输出向量，存储角度	输出

3.3.7.16 Rect

函数定义

```
void vrect_f(const int len, const float *src1, const float *src2, float _Complex *dst);
```

```
void vrect_d(const int len, const float *src1, const float *src2, float _Complex *dst);
```

函数说明：将输入极坐标转换为直角坐标。输出复数向量 dst 的实部表示横轴，虚部表示纵轴。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src1	单(双)精度浮点类型	输入向量，极坐标的半径	输入
src2	单(双)精度浮点类型	输入向量，极坐标的角度	输出
dst	复数单(双)精度浮点类型	输出向量，直角坐标	输出

3.3.7.17 Sumval

函数定义

```
double vsumval_d(const int len, const double *src);
```

```
float vsumval_f(const int len, const float *src);
```

函数说明：返回输入向量的所有元素的和。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
返回值	单(双)精度浮点类型	输出值	输出

函数定义

```
int vsumval_i(const int len, const int *src);
```

函数说明：返回输入向量所有元素的和。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	整型	输入向量	输入
返回值	整型	输出值	输出

3.3.7.18 Sumsqval

函数定义

```
double vsumsqval_d(const int len, const double *src);  
float vsumsqval_f(const int len, const float *src);  
double _Complex cvsumval_d(const int len, const double _Complex *src);  
float _Complex cvsumval_f(const int len, const float _Complex *src);
```

函数说明：返回输入向量所有元素平方的和。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点类型	输入向量	输入
返回值	(复数)单(双)精度浮点类型	输出值	输出

3.3.7.19 Meanval

函数定义

```
double vmeanval_d(const int len, const double *src);  
float vmeanval_f(const int len, const float *src);  
double _Complex cvmeanval_d(const int len, const double _Complex *src);  
float _Complex cvmeanval_f(const int len, const float _Complex *src);
```

函数说明：返回输入向量所有元素的平均值。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点类型	输入向量	输入
返回值	(复数)单(双)精度浮点类型	输出值	输出

3.3.7.20 Meansqval

函数定义

```
double vmeansqval_d(const int len, const double *src);
```

```
float vmeansqval_f(const int len, const float *src);
```

函数说明：返回输入向量所有元素的平方和的平均值。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
返回值	单(双)精度浮点类型	输出值	输出

函数定义

```
double cvmeansqval_d(const int len, const double _Complex *src);
```

```
float cvmeansqval_f(const int len, const float _Complex *src);
```

函数说明：返回输入向量所有元素的模的平方的和的平均值。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	复数单(双)精度浮点类型	输入向量	输入
返回值	单(双)精度浮点类型	输出值	输出

3.3.7.21 Modulate

函数定义

```
double cvmodulate_d(const int len, const double _Complex *src, const double nu,  
const double phi, double _Complex *dst);
```

```
float cvmodulate_f(const int len, const float _Complex *src, const float nu, const  
float phi, float _Complex *dst);
```

```
double vmodulate_d(const int len, const double *src, const double nu, const  
double phi, double _Complex *dst);
```

```
float vmodulate_f(const int len, const float *src, const float nu, const float phi, float  
_Complex *dst);
```

函数说明：通过指定的复频率计算实向量的调制。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点类型	输入向量	输入
nu	单(双)精度浮点类型数	输入值，每个指数的标量频率	输入

参数	类型	描述	输入/输出
		(以弧度为单位)	
phi	单(双)精度浮点类型数	输入值，以弧度为单位的标量初始阶段。	输入
dst	复数单(双)精度浮点类型	输出向量	输出
返回值	单(双)精度浮点数	返回值	输出

3.3.7.22 Cumsum

函数定义

```
void vcumsum_d(const int len, const double *src, double *dst);  
void vcumsum_f(const int len, const float *src, float *dst);
```

函数说明：计算向量元素的累积和。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输出向量	输出

函数定义

```
void vcumsum_i(const int len, const int *src, int *dst);
```

函数说明：计算整型向量元素的累积和。

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	整型	输入向量	输入
dst	整型	输出向量	输出

3.3.7.23 Lagr

函数定义

```
double vlagr_d(const int len, const double x,const double *src1, const double *src2,  
const unsigned int low,const unsigned int high);
```

函数说明：拉格朗日插值多项式。

参数说明

参数	类型	描述	输入/输出
----	----	----	-------

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
x	双精度浮点数	待求节点值	输入
src1	双精度浮点型	保存的已知节点值向量	输入
src2	双精度浮点数	保存对应已知节点的函数值向量	输入
low	无符号整型	索引	输入
ligh	无符号整型	索引	输入
返回	精度浮点数	返回待求节点的函数值	输出

3.3.7.24 Expoavg

函数定义

```
void vexpoavg_d(const int len, const double alpha, const double *src, double *dst);
void vexpoavg_f(const int len, const float alpha, const float *src, float *dst);
void cvexpoavg_d(const int len, const double alpha, const double _Complex *src,
double _Complex *dst);
void cvexpoavg_f(const int len, const float alpha, const float _Complex *src, float
_Complex *dst);
```

函数说明：计算输入两向量的加权平均值，输出为向量。

$$dst_i = \alpha * src_i + (1 - \alpha) * dst_i; \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
alpha	单(双)精度浮点数	src 向量元素的权重，标量	输入
src	单(双)精度浮点类型	输入向量	输入
dst	单(双)精度浮点类型	输入/输出向量	输入/输出

3.3.7.25 Ramp

函数定义

```
void vramp_d(const int len, const double x, const double y, double *dst);
void vramp_f(const int len, const float x, const float y, float *dst);
```

函数说明：通过从初始值开始并按步长大小递增来计算输出向量的每个元素。

$$dst_i = x + i * y; \quad 0 \leq i < len$$

参数说明：

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入

参数	类型	描述	输入/输出
x	单(双)精度浮点数	输入值, 矢量的初始值。	输入
y	单(双)精度浮点数	输入值, 矢量的增量	输入
dst	单(双)精度浮点类型	输出向量	输出

函数定义

```
void vramp_i(const int len, const int x, const int y, int *dst);
```

函数说明: 通过从初始值开始并按步长大小递增来计算输出向量的每个元素。

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
x	整型数	输入值, 矢量的初始值。	输入
y	整型数	输入值, 矢量的增量	输入
dst	整型数	输出向量	输出

3.3.7.26 Dot

函数定义

```
double vdot_d(const int len, const double *src1, const double *src2);
```

```
float vdot_f(const int len, const float *src1, const float *src2);
```

```
double _Complex cvdot_d(const int len, const double _Complex *src1, const
double _Complex *src2);
```

```
float _Complex cvdot_f(const int len, const float _Complex *src1, const float
_Complex *src2);
```

```
double _Complex cvjdot_d(const int len, const double _Complex *src1, const
double _Complex *src2);
```

```
float _Complex cvjdot_f(const int len, const float _Complex *src1, const float
_Complex *src2);
```

```
double _Complex rcvdot_d(const int len, const double *src1, const double
_Complex *src2);
```

```
float _Complex rcvdot_f(const int len, const float *src1, const float _Complex
*src2);
```

函数说明: 输出向量的点积分。

函数名	说明
cdot_f(_d)	单(双)精度实向量与实向量的点积
cvdot_f(_d)	单(双)精度复数向量与复数向量的点积

函数名	说明
cvjdot_f(_d)	单(双)精度复数向量与复数向量的共轭的点积
rcvdot_f(_d)	单(双)精度实向量与复数向量的点积

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	(复数)单(双)精度浮点类型	输入向量	输入
dst	(复数)单(双)精度浮点类型	输出向量	输出

3.3.7.27 Euler

函数定义

```
void veuler_d(const int len, const double *src, double _Complex *dst);
```

```
void veuler_f(const int len, const float *src, float _Complex *dst);
```

函数说明: 为向量的每个元素计算复平面中单位向量的角度对应的复数。

参数说明:

参数	类型	描述	输入/输出
len	整型数	向量元素个数	输入
src	单(双)精度浮点类型	输入向量	输入
dst	复数单(双)精度浮点类型	输出向量	输出

